

EviMind 项目解析教程

从启动、架构、核心链路到二次开发

2026 年 7 月 16 日

摘要

这是一份面向初学者的 EviMind 项目解析教程。读者不需要先熟悉 Spring Boot、Vue、RAG 和向量数据库的全部细节，只要按顺序阅读，就能理解项目定位、运行方式、目录结构、后端主链路、前端请求层、数据库模型、文档入库、RAG 问答、安全控制、部署方式和验证边界。本文基于当前仓库文件写成，避免脱离代码的泛泛介绍。

目录

1	先看项目定位	4
2	新手阅读路线	4
3	运行方式	4
3.1	开发模式	4
3.2	单 JAR 模式	5
3.3	Docker Compose 模式	5
4	技术栈总览	5
5	目录结构	6
6	后端启动入口	7
7	配置分层	7
7.1	默认配置	7
7.2	Standalone 配置	8
8	认证和权限主线	8
8.1	注册和登录	8
8.2	JWT 过滤器	9
8.3	业务权限	9

9	前端结构	9
9.1	请求封装	9
10	核心业务流程	10
10.1	总流程	10
10.2	知识库创建	10
11	文档上传和 ETL	10
11.1	上传入口	10
11.2	ETL 状态机	11
11.3	版本化入库	11
11.4	切片策略	12
12	检索链路	12
12.1	HybridSearchService	12
12.2	RRF 融合	12
12.3	Rerank 和证据门控	12
13	RAG 问答链路	13
13.1	ConversationController 到 RagPipeline	13
13.2	RagPipeline 做什么	13
13.3	证据组合	13
14	SSE 流式输出	14
15	数据库模型	14
15.1	核心表	14
15.2	读表技巧	15
16	文件分析和报告	15
17	前后端 API 对照	15
18	安全设计和剩余风险	16
18.1	已具备的安全点	16
18.2	需要继续加固的点	16
19	部署理解	17
19.1	Standalone	17
19.2	Docker Compose	17
20	工程验证	17

21 新手二次开发路线	18
21.1 加一个普通页面	18
21.2 加一个新文档格式	18
21.3 加一个检索策略	18
21.4 加一个大模型 provider	19
22 如何调试这个项目	19
22.1 调试上传	19
22.2 调试问答	19
22.3 调试前端	19
23 读完后的掌握标准	20
24 结语	20

1 先看项目定位

EviMind 是一个证据驱动的 RAG 知识工作台。它的核心目标不是单纯聊天，而是把文档上传、文本抽取、切片、检索、证据引用、研究笔记、引用导出和批量分析放进一个工作流。

一句话理解：

用户上传资料，系统把资料变成可检索证据；用户提问时，系统先检索证据，再让大模型基于证据回答，并返回引用。

当前项目适合三类场景：

- 本地研究资料管理：上传 PDF、Word、Markdown、代码、日志等文件，围绕知识库问答。
- RAG 工程学习：能看到文件入库、混合检索、RRF 融合、证据门控、SSE 流式输出的完整实现。
- 项目交付演示：支持 standalone、单 JAR 和 Docker Compose 三种运行模式。

2 新手阅读路线

不要从所有类开始看。这个项目模块多，直接翻目录容易乱。建议按下面顺序：

1. 先跑 standalone，确认登录、知识库、上传、问答四件事能串起来。
2. 看 README 的 Core Workflow，理解业务主线。
3. 看认证链路：JWT 过滤器、GroupContext、知识库成员校验。
4. 看文档上传链路：DocumentController 到 DocumentService，再到 EtlPipeline。
5. 看问答链路：ConversationController 到 ConversationService，再到 RagPipeline。
6. 看检索链路：HybridSearchService、PgVectorSearchService、ElasticsearchSearchService、RrfFusionService。
7. 最后看前端：路由、请求封装、ChatView、各模块 API 文件。

这样读的好处是先抓住数据流，再回头补框架细节。

3 运行方式

3.1 开发模式

开发模式使用 backend standalone profile，数据库走 H2，文件走本地目录。PostgreSQL、Elasticsearch、MinIO 都不是必需。

```
cd D:\EviMind
$env:JAVA_HOME = "C:\Program Files\Java\jdk-22"
$env:DEEPSEEK_API_KEY = "your-key"
.\mvnw.cmd spring-boot:run "-Dspring-boot.run.profiles=standalone"
```

前端单独启动：

```
cd D:\EviMind\frontend
npm install
npm run dev -- --host 127.0.0.1
```

浏览器打开：

```
http://127.0.0.1:5173
```

前端 Vite 会把 /api 代理到 `http://localhost:8080`。后端健康检查是：

```
http://localhost:8080/actuator/health
```

3.2 单 JAR 模式

单 JAR 模式适合本地交付。脚本会先构建前端，把 `frontend/dist` 复制到 `src/main/resources/static`，再打 Spring Boot JAR。

```
cd D:\EviMind
$env:JAVA_HOME = "C:\Program Files\Java\jdk-22"
$env:DEEPSEEK_API_KEY = "your-key"
.\build.bat
.\start.bat
```

访问入口：

```
http://localhost:8080
```

3.3 Docker Compose 模式

Docker Compose 模式启动 PostgreSQL/pgvector、Elasticsearch、MinIO、backend 和 frontend。它比 standalone 更接近完整部署环境。

```
cd D:\EviMind
copy .env.example .env
docker compose up -d
```

启动前必须在 `.env` 填入数据库、MinIO 和 JWT 密钥。Compose 缺少这些值会拒绝解析；后端和基础设施端口仅绑定到本机回环地址。

4 技术栈总览

层次

当前实现

后端框架	Spring Boot 3.5, Java 21, Spring Web, Spring Security, Actuator
AI 接入	Spring AI 1.0.0-M1, OpenAI-compatible ChatClient, 多 provider 配置
数据访问	MyBatis-Plus 为主, 部分 JPA Repository 用于分析结果
数据库	PostgreSQL/pgvector 用于完整栈, H2 用于 standalone
搜索	pgvector 语义检索、Elasticsearch 关键词检索、本地关键词回退
对象存储	MinIO 或本地文件存储
文档解析	PDFBox、Apache POI、OpenPDF、Tess4J、epub4j
前端	Vue 3, TypeScript, Vite, Element Plus, Pinia, Axios, Markdown-It
工程质量	Maven Surefire/Failsafe、JaCoCo、Spotless、PMD、GitHub Actions

初学者可以先不深究每个依赖。先记住：Spring Boot 负责 API, Vue 负责界面, PostgreSQL/pgvector 与 Elasticsearch 负责检索, MinIO/本地目录负责文件, Spring AI 负责模型调用。

5 目录结构

仓库主要目录如下：

```
EviMind/  
src/main/java/com/example/evimind/  
  assistant/      会话、流式对话、RAG 问答入口  
  auth/           注册、登录、JWT、刷新令牌  
  common/        健康检查、全局异常处理  
  config/         AI、安全、异步、存储、搜索配置  
  document/       文档上传、列表、删除、重试  
  extractor/      PDF、Word、Excel、PPT、EPUB、图片文字提取  
  ingestion/      ETL 入库流水线、切片、嵌入、ES 写入  
  knowledgebase/  知识库 CRUD、成员、检索入口  
  qa/            RAG 编排、证据组合、回答与引用  
  retrieval/      查询改写、向量检索、关键词检索、RRF  
  service/        分析、引用、报告、知识图谱等业务服务  
  storage/        MinIO 和本地文件存储  
src/main/resources/  
  application.yml  
  application-standalone.yml  
  schema-h2.sql
```

```
db/migration/  
prompts/  
frontend/  
src/views/  
src/stores/  
src/api/  
src/utils/
```

读目录时注意两个点：

- controller 和 assistant/document/knowledgebase 里都有 REST 入口，不能只看一个包。
- schema-h2.sql 服务 standalone；db/migration 服务 PostgreSQL/Flyway。两者覆盖范围不同。

6 后端启动入口

后端入口是 `EvimindApplication.java`。当前类排除了 Spring AI 的部分自动配置：

```
@SpringBootApplication(exclude = {  
    OpenAiAutoConfiguration.class,  
    ChatClientAutoConfiguration.class  
})
```

这表示项目自己创建 `ChatClient`，而不是完全依赖默认自动装配。具体实现看 `config/AiConfig.java`：

- 读取 `custom.ai.providers` 下的 `deepseek`、`zhipu`、`qianwen`、`openai` 配置。
- 为每个 provider 创建 OpenAI-compatible `OpenAiApi`。
- 生成 `Map<String, ChatClient>`，业务层按 provider 取模型。
- 如果开启 `embedding`，再创建 `OpenAiEmbeddingModel`。

当前检查点：`AiDataTools.java` 定义了 `listDirectory`、`readFileContent`、`queryAnalysisHistory` 三个函数 Bean，但当前 `AiConfig.java` 没有看到把这些函数显式绑定到 `ChatClient` 的调用。以后如果要做真正 agent 工具调用，需要补 `ChatClient` 函数绑定，并加测试验证。

7 配置分层

7.1 默认配置

`application.yml` 面向完整栈。它默认使用 PostgreSQL、Flyway、MinIO、Elasticsearch，`embedding` 默认开启，JWT secret 通过环境变量注入。

关键配置分组：

配置路径	作用
spring.datasource	PostgreSQL 连接
spring.flyway	数据库迁移
spring.elasticsearch. uris	Elasticsearch 地址
minio	对象存储
jwt	access token 和 refresh token 生命周期
custom.ai.providers	DeepSeek、GLM、Qianwen、OpenAI-style provider
custom.ai.embedding	向量模型配置
custom.rag	检索超时、查询改写、rerank、证据上下文长度
custom.analysis	批量分析和报告导出目录

7.2 Standalone 配置

application-standalone.yml 面向本地演示：

- 数据库：data/evimind-standalone.mv.db。
- 文件目录：data/documents。
- MinIO：关闭。
- Elasticsearch：默认关闭。
- Embedding：默认关闭。
- H2 Console：开启，路径是 /h2-console。

这能降低启动成本，但也带来边界：H2 schema 当前只覆盖核心表；完整 PostgreSQL 迁移还包含向量、引用、知识图谱、权限、审计、定时任务等表。做完整能力验证时应使用 Docker/production-like 栈。

8 认证和权限主线

8.1 注册和登录

认证入口在 AuthController.java，核心逻辑在 AuthService.java。

1. register：检查用户名唯一，BCrypt 加密密码，创建用户，返回 access token 和 refresh token。
2. login：校验用户名密码，禁用用户拒绝登录，返回新 token。
3. refresh：用 refresh token hash 查表，旧 refresh token 置为 revoked，再发新 token。
4. me：从 GroupContext 读取当前用户 ID，再查用户信息。

refresh token 不明文入库，入库的是 SHA-256 hash。这比直接保存 refresh token 更安全。

8.2 JWT 过滤器

`JwtAuthenticationFilter.java` 做三件事：

1. 从 Authorization header 提取 Bearer token。
2. 校验 token，取出 `userId`、`username`、`systemRole`。
3. 调用 `GroupService` 获取默认 `groupId`，把 `userId`、`groupId`、`role` 写进 `GroupContext`。

请求结束后，filter 在 finally 中清理 `GroupContext`。这个细节重要，因为 `GroupContext` 是 `ThreadLocal`，不清理会污染后续请求。

8.3 业务权限

后端权限不是只靠 `SecurityConfig`。很多业务服务会再次检查成员关系：

- `KnowledgeBaseService`：创建者是 OWNER；更新、删除、成员管理要求 OWNER。
- `DocumentService`：上传、列表、详情、删除、重试前检查知识库成员。
- `RagPipeline`：问答前检查用户是否是知识库成员。
- `ConversationService`：会话只能由 owner 访问。

新手调试 403 时，不要只看 Spring Security。更常见的原因是 `GroupContext` 没有用户，或 `kb_member` 没有对应记录。

9 前端结构

前端入口在 `frontend/src/main.ts`，路由在 `frontend/src/router/index.ts`。路由守卫逻辑很直接：需要登录的页面如果没有 `accessToken`，就跳到 `/login`。

主要页面：

页面	作用
<code>ChatView.vue</code>	证据问答、模型参数、SSE 流式输出、引用展示
<code>KnowledgeBaseView.vue</code>	知识库管理
<code>DocumentView.vue</code>	文档上传、列表、入库状态、重试
<code>AnalysisView.vue</code>	文件分析和批量分析
<code>CitationView.vue</code>	引用导出
<code>AcademicGraphView.vue</code>	学术引用图
<code>KnowledgeGraphView.vue</code>	知识图谱
<code>NotesView.vue</code>	研究笔记

9.1 请求封装

`frontend/src/utils/request.ts` 是前端理解后端的关键文件：

- Axios `baseUrl` 是 `/api/v1`。

- 每个请求自动带 Authorization header。
- 遇到 401 且未重试过，会调用 `/api/v1/auth/refresh` 刷新 access token。
- SSE 不能完全靠 Axios，所以提供 `authenticatedFetch` 给流式请求使用。这解释了为什么普通 API 用 `request.ts`，流式聊天用 `fetch` 读流。

10 核心业务流程

10.1 总流程

完整工作流如下：

```
register / login
-> create knowledge base
-> upload document
-> async ETL
-> create conversation
-> send question
-> hybrid retrieval
-> evidence-gated generation
-> stream answer + citations
```

这条线就是阅读项目的主干。

10.2 知识库创建

`KnowledgeBaseService.create` 会填默认值：

- `evidenceThreshold` 默认 0.50。
- `chunkStrategy` 默认 PARAGRAPH。
- `chunkSize` 默认 500。
- `chunkOverlap` 默认 100。
- 创建者写入 `kb_member`，角色是 OWNER。

知识库不是单纯分类目录，它还保存切片策略和证据阈值。也就是说，同一份文档放进不同知识库，后续入库和问答策略可能不同。

11 文档上传和 ETL

11.1 上传入口

文档上传入口是：

```
POST /api/v1/documents/upload
```

调用链：

```
DocumentController
-> DocumentService.upload
-> storage upload
-> insert document(status=UPLOADED)
-> triggerIngestionAsync
-> EtlPipeline.processDocument
```

DocumentService 会先做安全检查：

- 用户必须是知识库成员。
- 文件不能为空。
- 文件名会被清洗，路径穿越会被拒绝。
- 格式必须在白名单内。
- 文件不能超过 50MB。
- 常见扩展名和 MIME 类型要匹配。

存储路径不使用用户原始文件名，而是：

```
knowledgeBaseId/UUID/UUID.ext
```

原始文件名只作为展示元数据保存。

11.2 ETL 状态机

EtlPipeline 的状态顺序：

```
UPLOADED
-> EXTRACTING
-> CLEANING
-> CHUNKING
-> PERSISTING
-> EMBEDDING
-> INDEXING
-> ENRICHING
-> COMPLETED
```

失败时状态变为 FAILED，并记录 failedStage、errorCode、errorMessage、retryCount、startedAt、finishedAt、ingestionVersion。

11.3 版本化入库

项目使用 ingestionVersion 和 activeIngestionVersion 管理文档版本。重试入库时会产生新版本，只有新版本完成后才切换 activeIngestionVersion。检索时只读 active version，避免失败重试留下的旧 chunk 被召回。

这个设计的价值是：入库过程可以失败，但失败不应该污染线上可检索证据。

11.4 切片策略

`DocumentChunker.java` 支持三种策略：

策略	含义
<code>FIXED_LENGTH</code>	按固定长度切，简单但可能切断语义
<code>PARAGRAPH</code>	按段落组织，再受 <code>chunkSize</code> 和 <code>overlap</code> 约束
<code>SEMANTIC</code>	按句子聚合，更适合自然语言文本

`overlap` 不是越大越好。过大带来重复召回和成本上升；过小可能丢上下文。默认 500/100 是一个入门可用值。

12 检索链路

12.1 HybridSearchService

检索链路在 `HybridSearchService.java`：

```
user query
-> QueryRewriteService
-> pgvector semantic search
-> Elasticsearch keyword search
-> local keyword fallback
-> RRF fusion
```

两路检索通过 `CompletableFuture` 并行执行，默认后端超时是 1500ms。任一路失败都不会直接让整个 RAG 崩掉，而是降级到另一条路或本地关键词回退。

12.2 RRF 融合

RRF 是 Reciprocal Rank Fusion。项目里的 `RrfFusionService` 默认 `k=60`。核心思想是：结果在多个召回列表里排名越靠前，总分越高。

简化公式：

$$score(d) = \sum_{s \in S} \frac{1}{k + rank_s(d)}$$

当前实现还会先归一化各路分数，再把 rank confidence 和 source confidence 组合成最终置信度。这样能把语义检索和关键词检索合成一份统一结果。

12.3 Rerank 和证据门控

RAG 不是召回就回答。项目还有两个过滤步骤：

- Reranker: 如果开启并有可用实现, 对候选证据精排。
- Evidence Sufficiency Gate: 根据知识库 evidenceThreshold 判断证据是否足够。证据不足时, 系统返回不足提示, 而不是强行编答案。这是项目的核心价值之一。

13 RAG 问答链路

13.1 ConversationController 到 RagPipeline

流式问答入口:

```
POST /api/v1/conversations/{id}/messages/stream
```

调用链:

```
ConversationController.streamMessage
-> ConversationService.streamMessage
-> save user message
-> build recent conversation history
-> RagPipeline.streamQuery
-> save assistant message on complete
```

ConversationService 会保存用户消息, 再用最近 10 条消息构造 conversationHistory, 交给 QueryRewriteService 改写查询。流结束后, 它把助手完整回答和 citations 写入 message 表。

13.2 RagPipeline 做什么

RagPipeline 是 RAG 编排中心:

1. 检查用户是否是知识库成员。
2. 调用 HybridSearchService 检索证据。
3. 可选 rerank。
4. 判断证据是否充分。
5. 用 EvidencePortfolioSelector 选择证据组合。
6. 渲染 prompt 模板。
7. 选择 ChatClient。
8. 流式返回 token、citations、done 事件。

13.3 证据组合

EvidencePortfolioSelector.java 不是简单取 top N。它会综合:

- 置信度。
- 查询词覆盖。

- 文档多样性。
- 与已选证据的冗余度。
- 上下文长度预算。

这让最终 prompt 不只是高分片段堆叠，而是尽量覆盖不同证据来源。

14 SSE 流式输出

后端用 Reactor Flux 返回字符串事件。典型事件类型：

- token: 增量文本。
- citations: 引用列表。
- done: 流结束。
- error: 错误。

前端 ChatView.vue 通过 store 发起请求，使用 authenticatedFetch 读取流。这样可以同时支持 token 逐步渲染和 401 后刷新 token。

新手调试流式输出时按这个顺序看：

1. 浏览器 Network 是否收到 text/event-stream。
2. 后端是否进入 ConversationService.streamMessage。
3. RagPipeline 是否返回 token/citations/done。
4. 前端 store 是否把 token 追加到当前 assistant 消息。
5. citations 是否能被解析并显示在右侧证据栏。

15 数据库模型

15.1 核心表

standalone H2 的核心表：

表	作用
sys_user	用户账号、密码、角色、状态
sys_group	用户默认组和组织信息
group_member	用户和组的关系
knowledge_base	知识库配置、证据阈值、切片策略
kb_member	用户和知识库的成员关系
document	文档元数据、入库状态、版本信息
document_chunk	文档切片、chunk index、ingestionVersion
conversation	会话、绑定知识库、模型 provider
message	用户和助手消息、引用、工具调用记录
analysis_result	文件分析结果

refresh_token	refresh token hash、过期时间、撤销状态
research_note	研究笔记、高亮和标签

PostgreSQL 迁移还包含：

- document_chunk_embedding：向量数据。
- citation_link：引用关系。
- kg_entity、kg_relation：知识图谱。
- document_permission、audit_log：文档权限和审计。
- scheduled_task：定时任务。

15.2 读表技巧

这个项目读数据库不要从 SQL 开始背字段。按业务对象理解：

问题	主要字段或表
用户能看到什么	sys_user、sys_group、group_member、kb_member、document_permission
文档能否被问答使用	document.ingestion_status、document.active_ingestion_version、document_chunk.ingestion_version
回答能否溯源	message.citations、document、document_chunk
分析报告在哪里	analysis_result、custom.analysis.report-dir

16 文件分析和报告

项目除了 RAG，还有文件分析功能。相关入口在 AnalysisController.java，结果由 AnalysisResultService 保存。报告导出由 ReportExportService 完成，支持 Markdown 和 PDF。

报告输出目录来自：

```
custom.analysis.report-dir=.analysis-reports
```

PDF 导出使用 OpenPDF。中文字体优先使用配置的 pdf-font-path，找不到时尝试系统字体，最后回退到内置中文字体。

17 前后端 API 对照

模块	主要接口
----	------

Auth	POST/api/v1/auth/register, POST/api/v1/auth/login, POST/api/v1/auth/refresh, GET/api/v1/auth/me
Knowledge Base	GET/POST/api/v1/knowledge-bases, POST/api/v1/knowledge-bases/{id}/search, 成员管理接口
Documents	POST/api/v1/documents/upload, 列表、详情、chunks、retry、批量删除、批量重入库
Conversations	POST/api/v1/conversations, POST/api/v1/conversations/{id}/messages/stream, 导出、重命名、删除
Citations	POST/api/v1/citations/export
Notes	GET/POST/PUT/DELETE/api/v1/notes
Analysis	/api/analysis/batch, /api/analysis/results, Markdown/PDF 导出
Academic	引用图、引用统计、文献综述
Graph	知识图谱、实体邻居、路径查询

注意：有些旧接口在 /api 下，例如 /api/chat/stream 和 /api/models；主业务新接口大多在 /api/v1 下。

18 安全设计和剩余风险

18.1 已具备的安全点

- JWT stateless 认证。
- refresh token hash 入库并轮换。
- GroupContext 贯穿业务权限。
- 文档上传文件名清洗、格式白名单、MIME 检查、大小限制。
- 前端请求自动带 token，401 自动刷新。
- SecurityConfig 配置 CSP、CORS、无状态 session。
- AI 工具的 safePath 限制在 custom.data.base-dir 下，防止路径穿越。

18.2 需要继续加固的点

这些不是教程推测，而是当前代码和文档反映出的工程边界：

1. standalone H2 schema 覆盖核心表，未覆盖 PostgreSQL 迁移的全部能力；完整测试要用 Docker 栈。

2. `GlobalExceptionHandler.handleRuntime` 已返回通用错误信息且不记录异常原文；新增异常处理器时仍要避免把底层 `message` 返回给客户端。
3. Swagger、H2 Console、详细运行诊断和 `actuator` 指标已限制为管理员访问；H2 Console 仅在本地 `profile` 开启，生产仍需使用受控密钥和网络边界。
4. `AiDataTools` 已定义函数 `Bean`，但当前 `ChatClient` 绑定关系需要补验证；不要直接宣称 `agent` 工具已完整接通。
5. Docker Compose 要求显式提供敏感凭据，且默认仅暴露前端端口；它用于完整本地集成验证，不等同于生产部署。

19 部署理解

19.1 Standalone

Standalone 的目标是快：少依赖，适合演示、开发和离线交付。它牺牲了完整生产能力，尤其是 `pgvector`、`Elasticsearch`、`MinIO`、`Flyway` 迁移链。

适合：

- 新手首次启动。
- UI 演示。
- 不依赖向量检索的基本功能验证。

19.2 Docker Compose

Docker Compose 的目标是完整：`PostgreSQL/pgvector`、`Elasticsearch`、`MinIO` 和后端一起跑。它适合验证完整 RAG 栈、向量检索、对象存储和 `Flyway` 迁移。

适合：

- 接近生产的功能验证。
- 检查集成测试问题。
- 验证 `PostgreSQL` 与 `Elasticsearch` 的真实行为。

20 工程验证

本机无 Docker 时，后端本地门禁建议：

```
.\mvnw.cmd -q -DskipITs verify
```

完整门禁在 Docker 可用机器上跑：

```
.\mvnw.cmd verify
```

前端验证：

```
cd frontend
npm run build
```

常见问题:

现象	判断方式
JAVA_HOME not found	设置 JDK 21+, 本机 README 示例使用 JDK 22
full verify 报	当前机器没有 Docker 或 Docker 不可用, 不等于业
Docker/Testcontainers	务代码必然错
前端 401 循环	检查 refresh token 是否过期、localStorage 是否有
	旧 token
上传后一直 FAILED	看 document.failed_stage 和 error_message
问答没有引用	检查文档是否 COMPLETED, 检索是否返回结果,
	证据阈值是否过高
AI 不回答或模型不可用	检查 DEEPSEEK_API_KEY 或其他 provider key

21 新手二次开发路线

21.1 加一个普通页面

路线:

1. 在 frontend/src/views 新建页面。
2. 在 frontend/src/router/index.ts 注册路由。
3. 在 frontend/src/api 新增 API 封装。
4. 后端新增 Controller 和 Service。
5. 如果需要数据表, 加 Flyway migration, 同时考虑 H2 standalone 是否要补 schema。

21.2 加一个新文档格式

路线:

1. 在 DocumentService 的 ALLOWED_FORMATS 和 MIME 表里加入扩展名。
2. 在 extractor 包里实现或扩展 FileContentExtractor。
3. 补一个最小文件解析测试。
4. 上传样例文件, 确认 ETL 到 COMPLETED。

21.3 加一个检索策略

路线:

1. 新增 SearchService 或扩展已有检索服务。

2. 在 HybridSearchService 里加入召回结果。
3. 修改 RRF 融合输入或新增融合策略。
4. 给降级路径加测试，不要让某一路失败拖垮整个问答。

21.4 加一个大模型 provider

路线：

1. 在 application.yml 的 custom.ai.providers 下增加 provider。
2. 确认它兼容 OpenAI API 格式；不兼容就需要适配请求或响应。
3. 在前端模型选择中显示 provider。
4. 用 /api/models 检查配置是否读到。
5. 用 ChatView 发起一次流式问答。

22 如何调试这个项目

22.1 调试上传

按顺序看：

1. 前端上传请求是否命中 /api/v1/documents/upload。
2. 后端是否通过 kb_member 检查。
3. 文件格式和 MIME 是否通过。
4. document 是否插入，状态是否 UPLOADED。
5. EtlPipeline 是否被异步触发。
6. document.failed_stage 是否记录失败阶段。

22.2 调试问答

按顺序看：

1. 会话是否绑定 knowledgeBaseId。
2. 当前用户是否是知识库成员。
3. HybridSearchService 是否返回候选结果。
4. Reranker 是否异常降级。
5. evidenceThreshold 是否挡住回答。
6. ChatClient 是否存在。
7. SSE 是否返回 token、citations、done。

22.3 调试前端

按顺序看：

1. router 守卫是否因为 token 缺失跳转。

2. request.ts 是否带 Authorization header。
3. 401 后 refresh 是否成功。
4. store 状态是否更新。
5. 组件 computed 是否从 store 读到数据。

23 读完后的掌握标准

如果能回答下面问题，就说明已经真正理解项目：

1. 为什么上传后不是马上可问答，而要等 ingestion_status 到 COMPLETED？
2. 为什么检索要同时用 pgvector 和 Elasticsearch？
3. RRF 解决什么问题？
4. evidenceThreshold 过高会出现什么现象？
5. GroupContext 从哪里来，为什么 Service 层还要查 kb_member？
6. standalone 和 Docker Compose 的数据库能力有什么差异？
7. 前端为什么同时有 Axios 和 authenticatedFetch？
8. 如果要加一种文件格式，至少要改哪些地方？
9. 如果 full verify 失败，如何判断是代码问题还是 Docker/Testcontainers 环境问题？
10. 当前 agent function 定义和 ChatClient 绑定之间还需要检查什么？

24 结语

EviMind 的核心不是“套一个大模型接口”，而是把文档、权限、检索、证据、引用、会话、分析和部署放进一个可运行系统。初学者理解这个项目，要少背框架名，多追数据流：谁创建数据，谁改变状态，谁读取证据，谁负责兜底。把上传链路和问答链路看懂后，其他模块都能沿着同一套思路展开。